FIAP ON

BIG DATA

MODELOS RELACIONAIS

2

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

LISTA DE FIGURAS

Figura 1 – Representação gráfica de uma entidade.....7
Figura 2 – Representação gráfica de duas entidades.....7
Figura 3 – Relacionamento 1:1 8
Figura 4 – Relacionamento 1:N.....9
Figura 5 – Relacionamento N:N 10
Figura 6 – Representação de atributos em uma entidade.....10
Figura 7 – Exemplo de modelo relacional. 11
Figura 8 – Exemplo de entidade sem normalização.....16
Figura 9 – Aplicando a 1FN.....17
Figura 10 – Aplicando a 2FN.....18
Figura 11 – Aplicando a 3FN.....19

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

LISTA DE QUADROS

Quadro 1 – Nível de Isolamento e anomalias atendidas 14
Quadro 2 – Comandos DDL 20
Quadro 3 – Comandos DDL 21
Quadro 4 – Comandos TCL 21
Quadro 5 – Comandos DCL 21
Quadro 3 – Comandos DDL 22

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

LISTA DE COMANDOS DE PROMPT DO SISTEMA OPERACIONAL

Comando de prompt 1 – Aplicações que usam biblioteca a libwrap.so .....**Erro! Indicador não definido.**

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

SUMÁRIO

MODELOS RELACIONAIS 6
1 INTRODUÇÃO 6
2 ARQUITETURA RELACIONAL 6
3 NÍVEIS DE ISOLAMENTO 11
4 FORMAS NORMAIS 15
5 LINGUAGEM SQL.....19
REFERÊNCIAS.....23

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

MODELOS RELACIONAIS

1 INTRODUÇÃO

Em 1970, Edgar Codd propôs o modelo relacional, como uma nova maneira de representação de dados; na década de 80, o modelo relacional tornou-se padrão para a grande maioria dos Sistemas Gerenciadores de Banco de Dados (SGBD). Codd propôs que uma visão relacional dos dados permitiria a descrição dos dados de uma maneira natural, sem que sejam necessárias estruturas adicionais para sua representação, fornecendo maior independência dos dados em relação aos programas.

A ideia de modelo relacional baseia-se no princípio de que as informações em uma base de dados podem ser consideradas como relações matemáticas e que podem ser representadas através do uso de tabelas.

2 ARQUITETURA RELACIONAL

Para Puga (2015), a arquitetura dos SGBDs tem evoluído constantemente. Com essa evolução, os SGBDs passaram de sistemas fortemente integrados que atendiam uma única aplicação para sistemas modulares que permitem atender diversas aplicações e que usam a arquitetura cliente-servidor. O uso de SGBDs com arquitetura distribuída – que utilizam vários computadores para o armazenamento de dados – permite que trabalhem com grandes volumes de dados de um ambiente Big Data.

Uma das características fundamentais do uso de bancos de dados é que permitem a abstração dos dados, dessa forma, podemos ocultar detalhes do armazenamento de dados, que, geralmente, são supérfluos para a maioria dos usuários de bancos de dados.

Para Elmasri (2018), utilizamos um modelo de dados para fornecer significado a abstração de dados. Um modelo de dados dentro de um SGBD é um conjunto de conceitos que podem ser usados para descrever a estrutura de um banco de dados. Uma forma de representarmos esse modelo é através do Modelo Entidade Relacionamento (MER). O MER é um modelo conceitual utilizado na Engenharia de

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Software para descrever os objetos – chamados de entidades – envolvidos em um domínio de negócios, com suas características – chamadas de atributos – e como elas se relacionam entre si – chamados de relacionamentos. O MER representa esses objetos através de símbolos que facilitam a visualização e compreensão do modelo.

Uma entidade representa um objeto, um conceito, ou objeto do mundo real que são descritos dentro do banco de dados. Normalmente representamos uma entidade através de um retângulo.

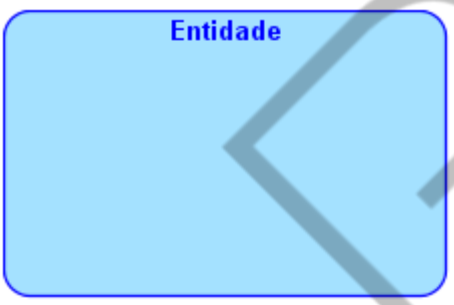


Figura 1 – Representação gráfica de uma entidade
Fonte: Elaborado pelo autor (2022)

Uma entidade é algo que pode ser definido e que pode ter dados armazenados sobre ele como uma pessoa, um objeto, conceito ou evento. As entidades são nomeadas com substantivos concretos ou abstratos que representem de forma clara sua função, por exemplo, Cliente, Aluno, Produto, Professor, Venda, Turma.

Figura 2 – Representação gráfica de duas entidades
Fonte: Elaborado pelo autor (2022)

A figura 2 mostra a representação gráfica de uma entidade denominada de “Empregado” e outra entidade chamada “Departamento”. No MER essa representação indica que armazenaremos informações sobre os empregados e sobre os departamentos onde trabalham. É importante ressaltar que as entidades serão transformadas em tabelas quando forem transcritas para o SGBD.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

As entidades podem atuar uma sobre as outras ou podem estar associadas uma com a outra, nesse caso, dizemos que uma entidade possui um relacionamento com outra entidade. A quantidade de objetos envolvidos em cada lado do relacionamento é chamada de cardinalidade. Podemos classificar os relacionamentos de três formas, segundo sua cardinalidade:

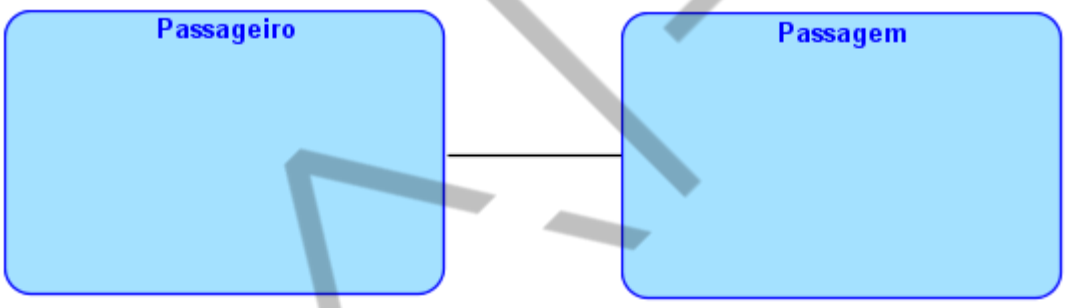
- **Relacionamento 1:1 (um para um):** cada uma das duas entidades envolvidas referenciam obrigatoriamente apenas uma unidade da outra. Por exemplo, em uma viagem interestadual, um passageiro compra uma passagem, a passagem comprada permite o embarque de apenas um passageiro.
- 

Figura 3 – Relacionamento 1:1
Fonte: Elaborado pelo autor (2022)

- Existem várias notações para indicar a cardinalidade. Na notação de Barker, utilizada nas figuras, o relacionamento 1:1 é representado por um traço simples ou acompanho por uma barra horizontal (|).
- **Relacionamento 1:N (um para muitos):** uma das entidades envolvidas pode referenciar várias unidades da outra, porém, do outro lado cada uma das várias unidades referenciadas só pode estar ligada uma unidade da outra entidade. Por exemplo, um departamento pode ter vários funcionários, no entanto, um funcionário só pode trabalhar em um departamento. É importante notar que só estão envolvidas duas entidades: funcionário e departamento. O que muda é a quantidade de unidades/exemplares envolvidas de cada lado.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

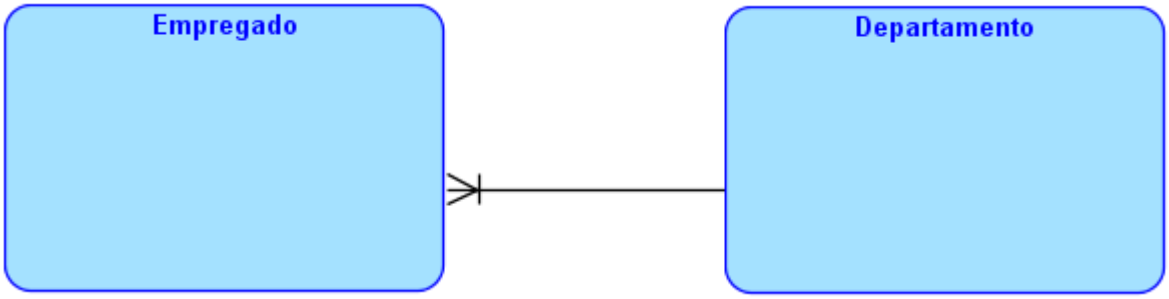
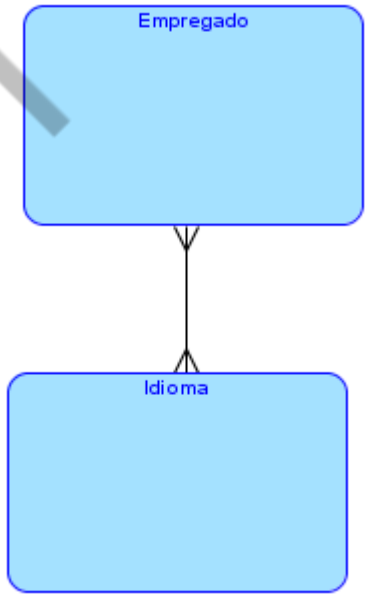


Figura 4 – Relacionamento 1:N
Fonte: Elaborado pelo autor (2022)

- Na notação de Barker, são usados três traços para indicar a cardinalidade muitos. Um traço horizontal, acompanhado de dois traços a 15 graus dele, formam uma figura que convencionou-se chamar de pé de galinha. A barra horizontal é lida como “um”. A figura 4 pode ser lida como “um departamento possui um ou mais empregados”.
- **Relacionamento N:N (muitos para muitos):** neste tipo de relacionamento cada entidade, de ambos os lados, pode referenciar múltiplas unidades da outra. Por exemplo, um empregado pode falar vários idiomas e um idioma pode ser falado por vários empregados. Nesse caso, um objeto do tipo empregado pode referenciar múltiplos objetos do tipo idioma e um objeto do tipo idioma pode referenciar múltiplos objetos do tipo empregado.
- 

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Figura 5 – Relacionamento N:N
Fonte: Elaborado pelo autor (2022)

É comum que os relacionamentos sejam expressos com verbos ou expressões que representam a ação que uma entidade exerce sobre a outra ou como elas interagem. A forma de leitura pode variar de acordo com a direção em que se lê o relacionamento. Por exemplo: um aluno assiste várias aulas, enquanto uma aula é assistida por vários alunos.

As entidades também possuem atributos. Um atributo é uma característica ou propriedade de uma entidade. Por exemplo, um empregado possui, entre outros, os atributos nome, endereço e salário. Os atributos são classificados como simples, composto, derivado, bem como de valor único ou multivalor. Simples: significa que o valor do atributo é atômico e não pode ser dividido, como um número de telefone. Composto: subatributos nascem de um atributo. Derivado: o atributo é calculado, ou derivado de outro atributo, tal como a idade a partir de uma data de nascimento.

Figura 6 – Representação de atributos em uma entidade.
Fonte: Elaborado pelo autor (2022)

O exemplo da figura 6 mostra a entidade empregado com os atributos nome, endereço e salário.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

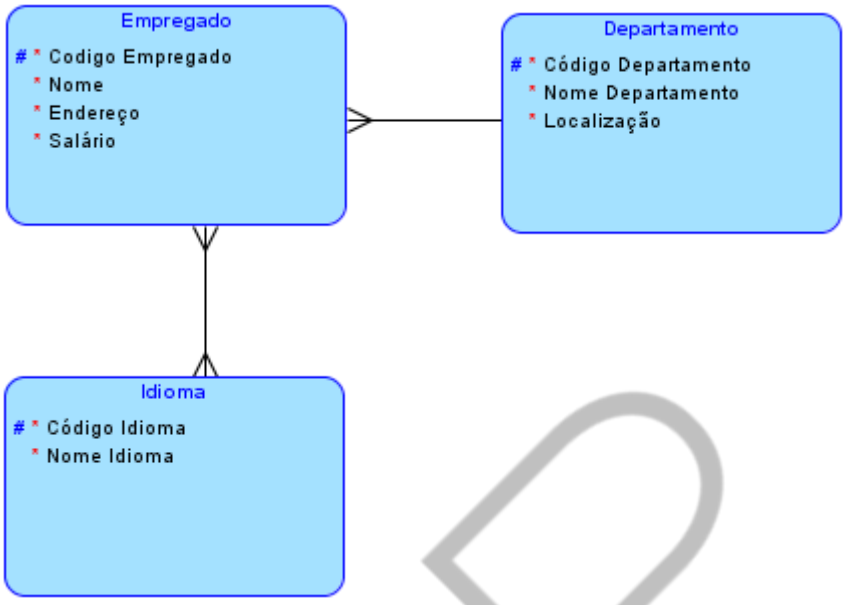


Figura 7 – Exemplo de modelo relacional.
Fonte: Elaborado pelo autor (2022)

O diagrama da Figura 7 pode ser lido como um departamento possui muitos empregados e um empregado trabalha em um departamento. Empregado fala vários idiomas e idioma é falado por vários empregados. Um departamento possui os atributos código departamento, nome departamento e localização. Empregado possui os atributos código empregado, nome, endereço e salário. Idioma possui os atributos código idioma e nome idioma. O modelo relacional nos fornece uma visão simplificada do funcionamento do nosso negócio.

3 NÍVEIS DE ISOLAMENTO

Segundo a Oracle (2020), uma transação é uma unidade de trabalho lógica e atômica que contém uma ou mais instruções SQL. Uma transação agrupa instruções SQL para que sejam todas confirmadas, ou seja, aquele grupo de instruções será aplicado ao banco de dados, ou todas revertidas, ou seja, as ações sobre o banco de dados executadas por aquele grupo de instruções serão desfeitas.

O uso de transações é uma das maneiras mais importantes pelas quais um sistema de gerenciamento de banco de dados difere de um sistema de arquivos. Todas as transações obedecem às propriedades básicas de uma transação de banco de dados, conhecidas como propriedades ACID. ACID é um acrônimo para Atomicidade, Consistência, Isolamento e Durabilidade.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

- Atomicidade: Todas as tarefas de uma transação são executadas ou nenhuma delas é. Não há transações parciais. Por exemplo, se uma transação começar a atualizar 1000 linhas, mas, por qualquer motivo, o sistema falhar após 200 atualizações, o banco de dados reverterá as alterações em todas essas 200 linhas.
- Consistência: A transação leva o banco de dados de um estado consistente para outro estado consistente. Por exemplo, em uma transação bancária que debita uma conta poupança e credita uma conta corrente, uma falha não deve fazer com que o banco de dados credite apenas uma conta, o que levaria a dados inconsistentes.
- Isolamento: O efeito de uma transação não é visível para outras transações até que a transação seja confirmada. Por exemplo, um usuário que atualiza a tabela aluno não vê as alterações não confirmadas nos alunos feitas simultaneamente por outro usuário. Assim, parece aos usuários como se as transações estivessem sendo executadas em série.
- Durabilidade: As alterações feitas por transações confirmadas são permanentes. Após a conclusão de uma transação, o banco de dados garante, por meio de seus mecanismos de recuperação, que as alterações da transação não sejam perdidas.

Mesmo que existam muitas transações simultâneas sendo executadas, a capacidade de isolamento de um banco de dados permite que uma transação seja executada como se não houvesse outras transações em execução simultaneamente.

O objetivo geral é evitar leituras e gravações de dados temporários, abortados ou incorretos gravados por transações simultâneas. O nível de isolamento é definido pelos seguintes fenômenos:

- **Leituras sujas ou *dirty reads*:** Em uma transação, os dados que ainda não foram confirmados são chamados de “sujos”. Em uma leitura suja, uma transação lê os dados escritos e ainda não confirmados por uma transação concorrente. Por exemplo, a transação X atualiza uma linha e não confirma a alteração, em seguida a transação Y lê os dados

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

atualizados. Se a transação X reverter a atualização, a transação Y terá lido dados que são considerados como inexistentes.

- **Leitura não repetível ou *non-repeatable reads***: Uma transação relê os dados que leu anteriormente e percebe que os dados foram modificados por outra transação, isto é, os dados foram alterados e confirmados desde a leitura inicial. A diferença entre a leitura não repetível e a leitura suja é que a outra transação foi confirmada, além disso, são necessárias duas leituras para que esse fenômeno se manifeste. Por exemplo, a transação X lê os dados, devido à simultaneidade, a transação Y atualiza dos dados lidos e efetua a confirmação. Se a transação X reler os mesmos dados, obterá um valor diferente.
- **Leituras fantasmas ou *phantom reads***: Uma transação reexecuta uma consulta retornando um conjunto de linhas que atendem a uma condição de pesquisa e descobre que o conjunto de linhas que atende à condição foi alterado devido a outra transação confirmada recentemente. A diferença entre a leitura não repetível e a leitura fantasma é que esta envolve uma coleção de alteração que atende um predicado de consulta ou invés de um único ítem. Por exemplo, a transação X recupera um conjunto de linha que atenda um determinado critério de pesquisa, em seguida, a transação Y gera novas linhas que correspondem aos critérios de pesquisa da transação X. Se a transação X executar a instrução que recupera o conjunto de linhas novamente, obterá um conjunto de linhas diferentes.

Com base nesses fenômenos, foram definidos quatro níveis de isolamento:

- *Read Uncommitted* (UR) é o nível de isolamento mais baixo. Esse nível garante que nenhuma transação possa atualizar uma linha do banco de dados se outra transação já a atualizou e ainda não foi confirmada. Isso impede que dois usuários atualizem o mesmo dado ao mesmo tempo, mas não impede leituras sujas.
- *Read Committed* (RC) esse nível de isolamento não permite que nenhuma outra transação grave ou leia uma linha na qual outra

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

transação tenha gravado, mas ainda não confirmada. Assim, não permite leitura suja. A transação mantém um bloqueio de leitura ou gravação na linha atual e, portanto, impede que outras transações a leiam, atualizem ou excluam.

- *Repeatable Read* (RR) esse nível de isolamento garante que qualquer transação que leia dados de uma linha impeça que outras transações de gravação acessem a mesma linha. Este é o nível de isolamento mais restritivo que mantém bloqueios (*locks*) de leitura em todas as linhas às quais faz referência e bloqueia todas as linhas que insere, atualiza ou exclui. As demais transações não podem ler, atualizar ou excluir as linhas bloqueadas, e, em consequência, a leitura não repetível é evitada.
- *Serializable* (S) esse é o mais alto nível de isolamento mais alto, ele é usado quando é necessário bloquear toda a tabela, impedindo que outras transações insiram ou leiam dados dela, dando a impressão que todas as transações sendo executadas concorrentemente pareçam estar sendo executadas em série.

Nível de Isolamento	Leituras sujas (*dirty reads*)	Leitura não repetível (*non-repeatable reads*)	Leituras fantasmas (*phantom reads*)
UR	Pode ocorrer	Pode ocorrer	Pode ocorrer
RC	Não ocorre	Pode ocorrer	Pode ocorrer
RR	Não ocorre	Não ocorre	Pode ocorrer
S	Não ocorre	Não ocorre	Não ocorre

Quadro 1 – Nível de Isolamento e anomalias atendidas
Fonte: Elaborado pelo autor baseado em Elmasri (2018)

A escolha do nível de isolamento depende única e exclusivamente dos requisitos de negócio da aplicação. Por exemplo, em aplicações tradicionais, não é esperado que uma transação leia dados alterados e ainda não confirmados por outras transações, sendo assim, o isolamento mínimo, nesta situação, é o *Read Committed*, já em aplicações que leem dados que não são alterados, o isolamento mínimo, nesta situação, é o *Read Uncommitted*.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

4 FORMAS NORMAIS

Para Elmasri (2018), a normalização de dados é o “processo formal passo a passo que examina os atributos de uma entidade, com o objetivo de evitar anomalias observadas na inclusão, exclusão e alteração de registros”.

Em SGBDs relacionais, por questões de desempenho, armazenamento, organização e facilidade de uso, evita-se misturar assuntos dentro de uma mesma entidade. Por exemplo: em uma entidade Empregado devem existir somente atributos relacionados com o assunto Empregado. Não se deve misturar atributos relacionados com outros assuntos, tais como Departamento e Idioma. O objetivo é evitar a redundância desnecessária dos dados bem como inconsistência dos dados.

Os principais objetivos da normalização de dados são:

- Objetivos;
    - Minimizar redundâncias e inconsistências de dados;
    - Facilitar a manipulação do banco de dados;
    - Facilitar a manutenção dos sistemas de informação.

É comum que, após as regras de normalização de dados serem aplicadas, algumas entidades sejam partidas em duas ou mais entidades, gerando um número maior de entidades no modelo. Este processo causa a simplificação dos atributos de uma entidade, colaborando significativamente para a estabilidade do modelo de dados, reduzindo-se consideravelmente as necessidades de manutenção.

As formas normais são importantes instrumentos para resolver antecipadamente problemas na estrutura do banco de dados. Apesar de existirem várias formas normais, é prática do mercado que utilizemos apenas a Primeira Forma Normal (1FN), a Segunda Forma Normal (2FN) e a Terceira Forma Normal (3FN).

As formas normais são aplicadas sequencialmente, isto é, primeiro aplica-se a 1FN, em seguida, se todas as entidades estiverem na 1FN, aplica-se a 2FN e, finalmente, se todas as entidades estiverem na 2FN, aplica-se a 3FN.

Segundo Elmasri (2018): “uma relação estará na Primeira forma normal 1FN, se e somente se todos os domínios básicos contiverem somente valores atômicos”,

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

isto é, não contiver grupos repetitivos. A Primeira Forma Normal não admite repetições ou campos que tenham mais que um valor, os procedimentos para aplicar a primeira forma normal são:

- Identificar a chave primária da entidade;
- Identificar o grupo repetitivo e removê-lo da entidade;
- Criar uma nova entidade com a chave primária da entidade anterior e o grupo repetitivo.

A chave primária da nova entidade será obtida pela concatenação da chave primária da entidade inicial e um ou mais atributos do grupo repetitivo.

Considere a seguinte entidade não normalizada:

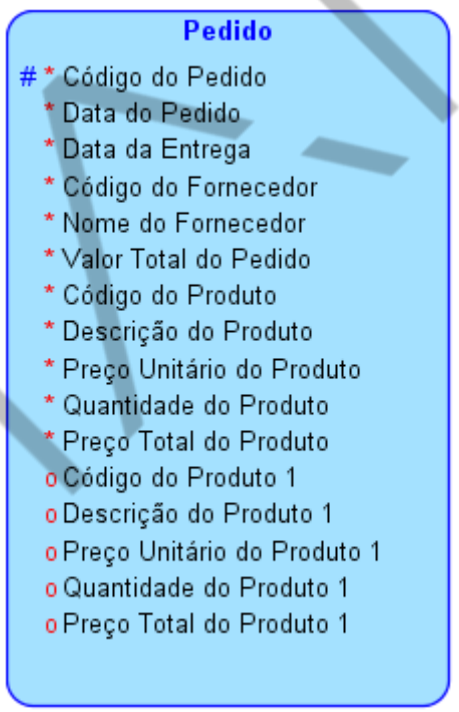


Figura 8 – Exemplo de entidade sem normalização
Fonte: Elaborado pelo autor (2022)

O exemplo da Figura 8 mostra uma entidade fora das formas normais. O primeiro passo para aplicar a primeira forma normal é identificar a chave primária da entidade, no caso, o atributo Código do Produto é a chave primária da entidade.

O segundo passo é identificar os atributos repetitivos e removê-los da entidade. Observando os atributos, é possível notar que alguns deles aparecem mais de uma

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

vez: código do produto, descrição do produto, preço unitário do produto e preço total do produto. Os atributos que aparecem mais de uma vez caracterizam um grupo repetitivo.

O terceiro passo é criar uma nova entidade com a chave primária da entidade anterior e o grupo repetitivo. Dessa forma, ao ser aplicada a 1FN no exemplo, o modelo passa a ter mais uma entidade e a entidade original fica mais simples de ser trabalhada.

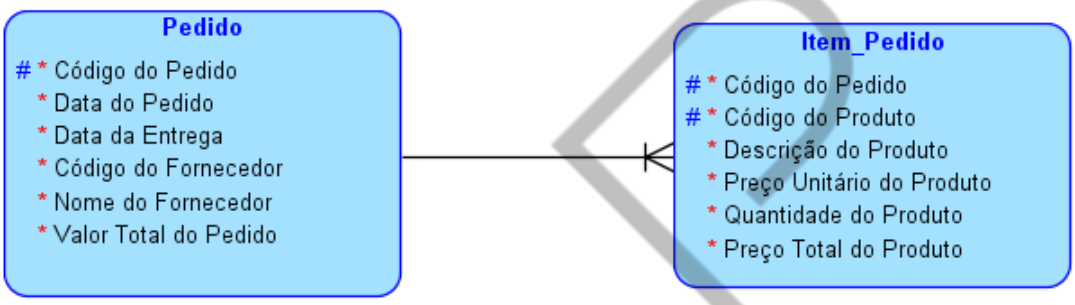


Figura 9 – Aplicando a 1FN
Fonte: Elaborada pelo autor (2022)

De acordo com Elmasri (2018): “Uma tabela está na Segunda Forma Normal se ela estiver na Primeira Forma Normal e todos os atributos não chave forem totalmente dependentes da chave primária”, isto é, o atributo depende de toda a chave e não apenas de parte dela. A dependência parcial é uma situação particular em que os atributos não chaves dependem parcialmente de um atributo da chave composta. Os procedimentos para aplicar a segunda forma normal são:

- Identificar os atributos que possuem dependência parcial.
- Remover da entidade todos esses atributos identificados e criar uma nova entidade com eles.
- A chave primária da nova entidade será o atributo do qual os atributos removidos são funcionalmente dependentes.

Examinando as entidades, é possível identificar que a entidade Item_Pedido da Figura 9 é a única entidade que possui uma chave primária composta, quer dizer, uma chave primária com mais de um atributo, sendo assim, é nessa entidade que deve ser aplicada a segunda forma normal.

Começamos identificando os atributos que possuem dependência parcial. O atributo Descrição do Produto depende funcionalmente do atributo Código do Produto,

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

mas não depende do atributo Código do Pedido, portanto, não está na segunda forma normal. Isto gera problemas com a manutenção dos dados, pois se houver alteração no nome do produto teremos que alterar em todos os registros de Item_Pedido.

Para normalizar a entidade, será necessário criar uma nova entidade que conterá os atributos Código do Produto e Descrição do Produto. Os demais atributos permanecem em Item_Pedido pois dependem tanto do Código do Pedido quanto do Código do Produto.

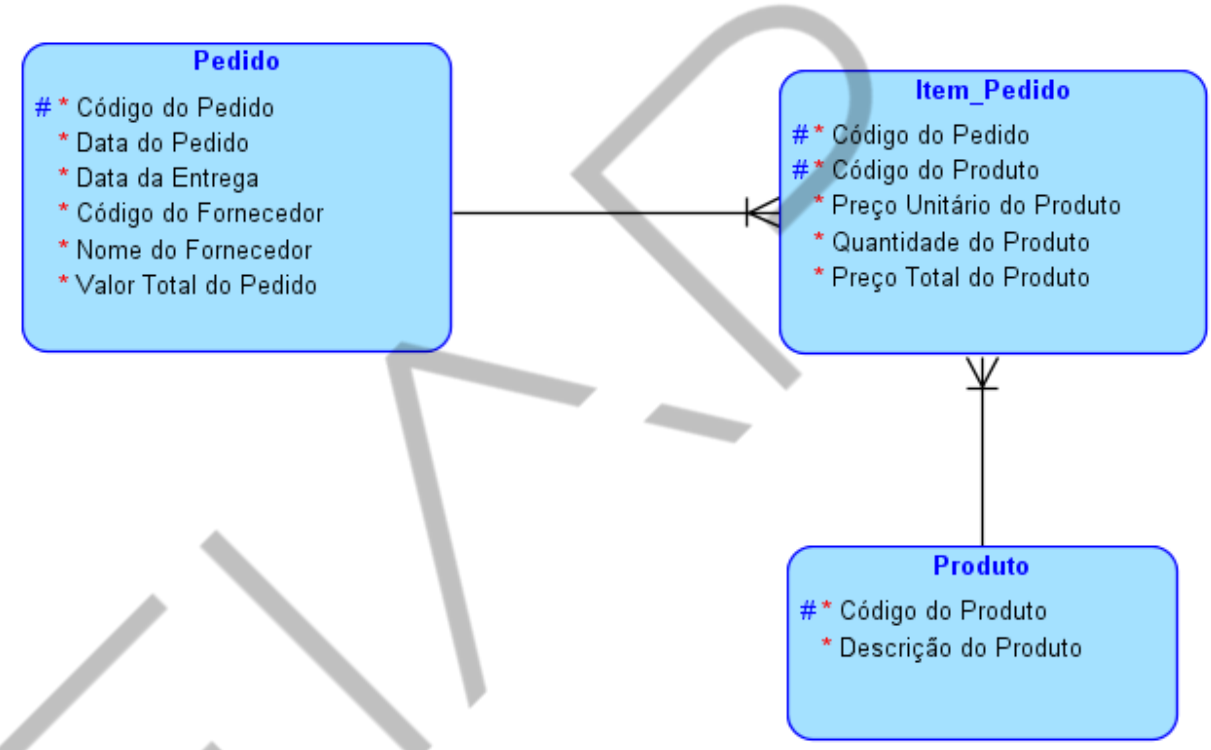


Figura 10 – Aplicando a 2FN
Fonte: Elaborado pelo autor (2022)

Uma tabela está na Terceira Forma Normal 3FN se ela estiver na 2FN e se nenhuma coluna não-chave depender de outra coluna não-chave. Na terceira forma normal temos de eliminar aqueles campos que podem ser obtidos pela equação de outros campos da mesma entidade, chamamos essa dependência de dependência transitiva. Os procedimentos para aplicar a terceira forma normal são:

- Identificar todos os atributos que possuem dependência transitiva;
- Removê-los e criar uma nova entidade com os mesmos;
- A chave primária da nova entidade será o atributo do qual os atributos removidos são funcionalmente dependentes.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Observando a entidade Pedido na Figura 10, é possível notar que o atributo Nome do Fornecedor depende do atributo Código do Fornecedor, mas não depende do atributo Código do Pedido. Para normalizar a entidade, será necessário criar uma nova entidade que conterá os atributos Código do Fornecedor e Nome do Fornecedor. Os demais atributos permanecem em Pedido, pois dependem apenas do atributo Código do Pedido.

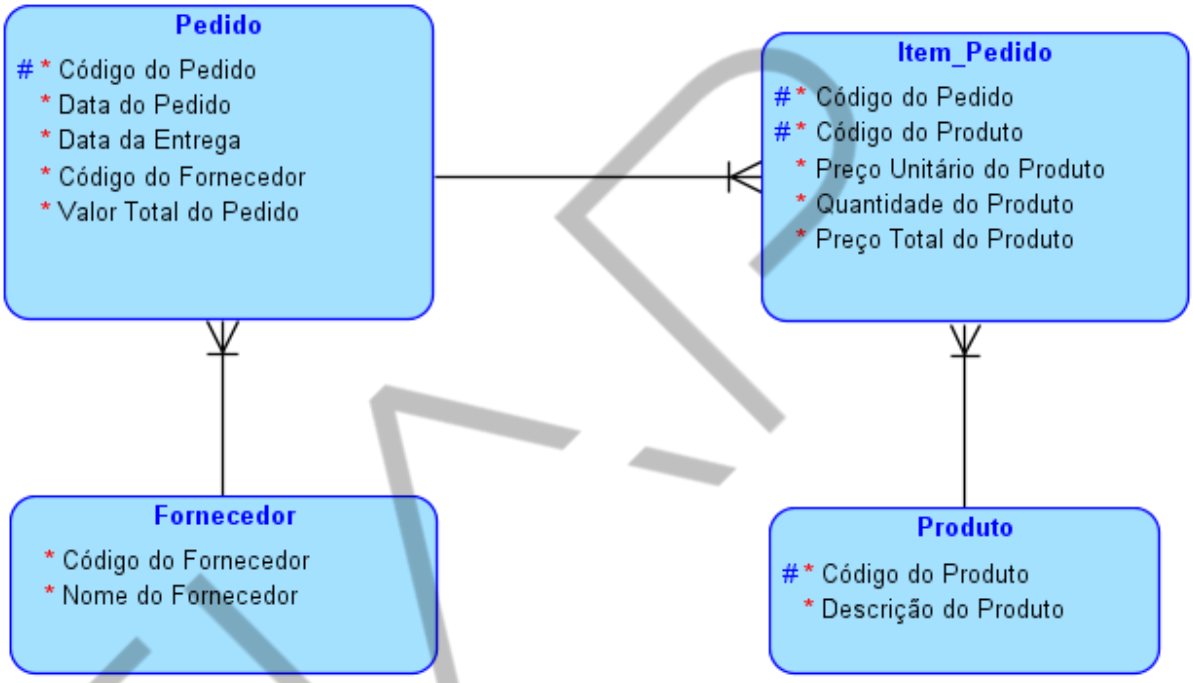


Figura 11 – Aplicando a 3FN
Fonte: Elaborado pelo autor (2022)

As regras para normalização de dados aplicadas de forma correta contribuem consideravelmente para a criação de uma boa estrutura das bases de dados relacionais, evitando anomalias de redundância ou perda de informação.

5 LINGUAGEM SQL

Para Oracle (2020), Structure Query Language (SQL) é uma linguagem de consulta de banco de dados usada para armazenar e gerenciar dados em SGBDs Relacionais. SQL foi a primeira linguagem comercial introduzida para o modelo relacional de banco de dados de E.F Codd. Hoje, quase todos os RDBMS (MySql, Oracle, Infomix, Sybase, MS Access) usam SQL como a linguagem de consulta de

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

banco de dados padrão. SQL é usado para executar todos os tipos de operações de dados em RDBMS.

SQL é dividido em cinco linguagens:

- Data Definition Language (DDL) ou linguagem de definição de dados trata das alterações na estrutura da tabela, como criação de tabela, alteração de tabela, exclusão de uma tabela. Todos os comandos DDL são confirmados automaticamente. Isso significa que ele salva todas as alterações permanentemente no banco de dados.

Comando	Descrição
• CREATE	• Cria uma nova tabela ou banco de dados.
• ALTER	• Altera a estrutura de uma tabela ou objeto do banco dados.
• TRUNCATE	• Exclui definitivamente os dados de uma tabela de um banco de dados.
• DROP	• Elimina uma tabela ou objeto do banco de dados.
• RENAME	• Renomeia uma tabela ou objeto do banco de dados.

Quadro 2 – Comandos DDL
Fonte: Elmasri (2018)

- Data Manipulation Language (DDL) ou linguagem de manipulação de dados são comandos usados para manipular os dados armazenados na tabela e não a própria tabela. Os comandos DML não são confirmados automaticamente. Isso significa que as alterações não são permanentes no banco de dados e elas podem ser revertidas.

• Comando	• Descrição.
• INSERT	• Insere uma nova linha na tabela.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

• UPDATE	• Atualiza os dados de uma ou mais linhas na tabela.
• DELETE	• Apaga uma ou mais linhas da tabela.

Quadro 3 – Comandos DDL
Fonte: Elmasri (2018)

- Transaction Control Language (TCL) ou linguagem de controle de transações são comandos usados para reverter ou confirmar as ações efetuadas pelos comandos DDL, manipular os dados armazenados na tabela e não a própria tabela. Os comandos DML não são confirmados automaticamente. Isso significa que as alterações não são permanentes no banco de dados e elas podem ser revertidas.

• Comando	• Descrição.
• COMMIT	• Grava as alterações efetuadas.
• ROLLBACK	• Desfaz as alterações efetuadas.
• SAVEPOINT	• Cria um ponto de salvamento temporário.

Quadro 4 – Comandos TCL
Fonte: Elmasri (2018)

- Data Control Language (DCL) ou linguagem de controle de dados são comandos usados para conceder ou revogar privilégios a usuários do banco de dados.

• Comando	• Descrição.
• GRANT	• Concede um privilégio.
• REVOKE	• Revoga um privilégio.

Quadro 5 – Comandos DCL
Fonte: Elmasri (2018)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

- Data Query Language (DQL) ou linguagem de consulta de dados é usada para obter dados de tabelas com base nas condições definidas pelo usuário.

• Comando	• Descrição.
• SELECT	• Obtêm dados de uma ou mais tabelas.

Quadro 6 – Comandos DDL
Fonte: Elmasri (2018)

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

REFERÊNCIAS

PUGA, S.; FRANÇA, E.; GOYA, M. **Banco de Dados**. São Paulo: Pearson, 2015.

ELMASRI, R.; NAVATHE, S.; HAMKANT, B. **Sistemas de banco de dados**. São Paulo: Pearson, 2018.

Oracle. **Database Concepts**. 2020. Disponível em: <https://docs.oracle.com/database/121/CNCPT/transact.htm#CNCPT016>. Acesso em: 21 jan. 2021.

CODD, E. F. A Relational Model of Data for Large Shared Data Banks, **Communications of the ACM**. Volume 13, nº 6, Junho de 1970, p. 377-387.

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com